Introducción a Python

MATERIAL 1

TEMA 1: ¿QUÉ ES PYTHON Y CÓMO FUNCIONA?

Material práctico

Autor:

Alexis Adonai Morales Alberto

2026

fancyhdr

Índice

1. Introducción **2**

2. Instalación de Python **2**
2.1. Instalación del lenguaje Python 3

3. Instalación de Anaconda **4**

4. ¿Cómo se instala una paquetería en Python? **7**
4.1. Instalacion de paqueterías mediante CMD 7
4.2. Instalando paquetes mediante Anaconda 8

5. ¿Cómo se importa una librería para trabajar en una sesión? **10**
5.1. Importando una función de un modulo 10

6. Operaciones básicas **11**
6.1. Suma 11
6.2. Resta 12
6.3. Multiplicación 12
6.4. División 12

7. Creación de objetos **13**

8. Tipos de objetos **13**
8.1. Array 14
8.2. Data frame 14
8.3. List 15
8.4. Tupulas 15
8.5. Funciones 16
8.6. Sustracción de elementos en los diferentes tipos de objetos 16

9. Vectores y matrices **17**
9.1. Generando vectores 17
9.2. Generando matrices 18
9.2.1. Operaciones con matrices 18

1. Introducción

En este documento se mostrará la manera en cómo se puede instalar Python mediante dos maneras:

1. Desempaquetando el lenguaje.
2. Instalandolo mediante Anaconda.

Además se mostrará qué son las librerías, cómo se instalan y la utilidad que tienen las librerías, así como las operaciones básicas que se pueden realizar mediante Python tales como:

1. Resta.
2. Suma.
3. Multiplicación.
4. División.

También podrás aprender diversas maneras de cómo construir diferentes objetos que van a ser de utilidad a lo largo de la visualización de datos, los cuales se enlistan a continuación.

1. Variables.
2. Matrices.
3. Arrays.
4. Data frame.

2. Instalación de Python

Para realizar la instalación de Python, se tendrán que descargar los archivos necesarios para elaborar todo este procedimiento. Cabe mencionar que solo se mostrará esta opración con el sistema operativo de Windows.

Primero, lo que se debe hacer es descargar tanto Python (como lenguaje) y el IDE de Anaconda, los cuales se anexan a continuación:

1. Python: https://www.python.org/downloads/

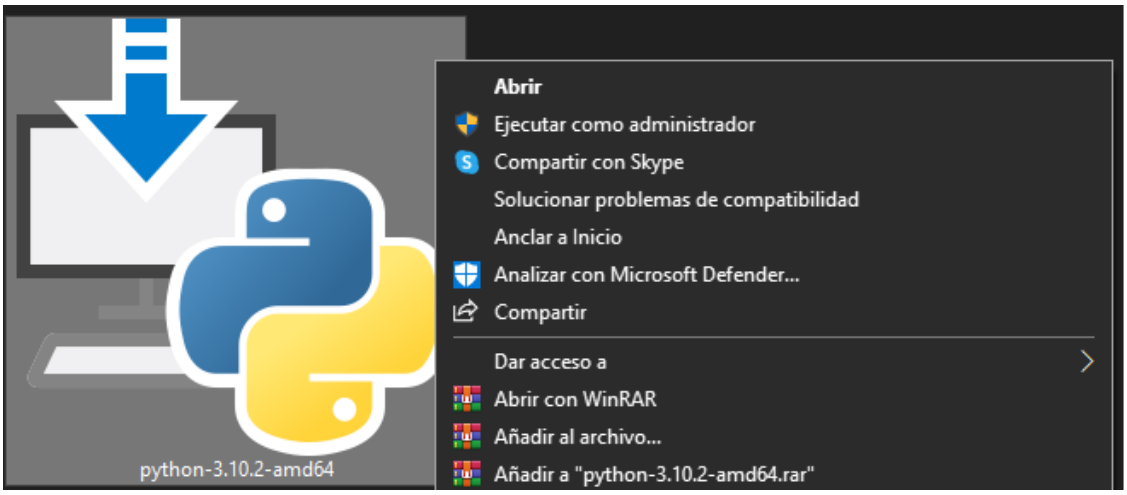
2. Anadona: https://www.anaconda.com/products/individual

Posteriormente, se mostrará la instalación de cada unno de los paquetes.

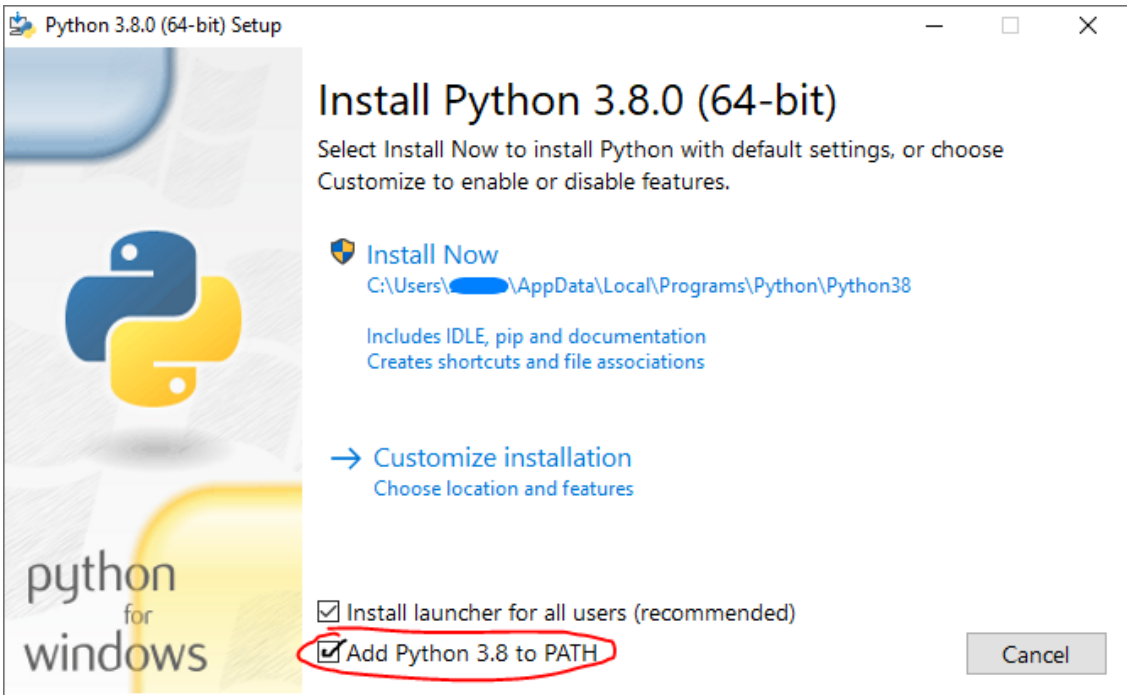
2.1. Instalación del lenguaje Python

El lenguaje de Python (sin algún IDE o interfaz) se puede instalar para poder asociarlo como Kernel a cualquier otra interfaz de usuario, tales como: RStudio, Visual Studio, etc. Es por ello que se debe explicar la instalación y el proceso para poder realizaro.

Como un primer paso, se ejecutará el archivo *python-x.xx.amdxx.exe* como administrador, tal y cómo se muestra en la siguiente imagen:



Posterior a ello, se empezará a ejecutar el instalador. Es importante qué en la ventana se marque la casilla **add Python x.xx to path** para que este se instale de la forma adecuada y no se presenten problemas si se quiere enlazar como kernel con otras IDE o interfaz de usuario.



Después de ello, se selecciona **install now** para que se ejecute el proceso de instalación. El resto solo es marcar *siguiente* y con ello Python quedará instalado. Para comprobrar que Python se encuentra corretamente instalado, se ejcutará CMD *símbolo de sistema* y se escribirá el siguiente comando: **python –version** y deberá mostrarse como resultado la versión de Python.

```
C:\Users\Morales>python --version
Python 3.9.7
```

3. Instalación de Anaconda

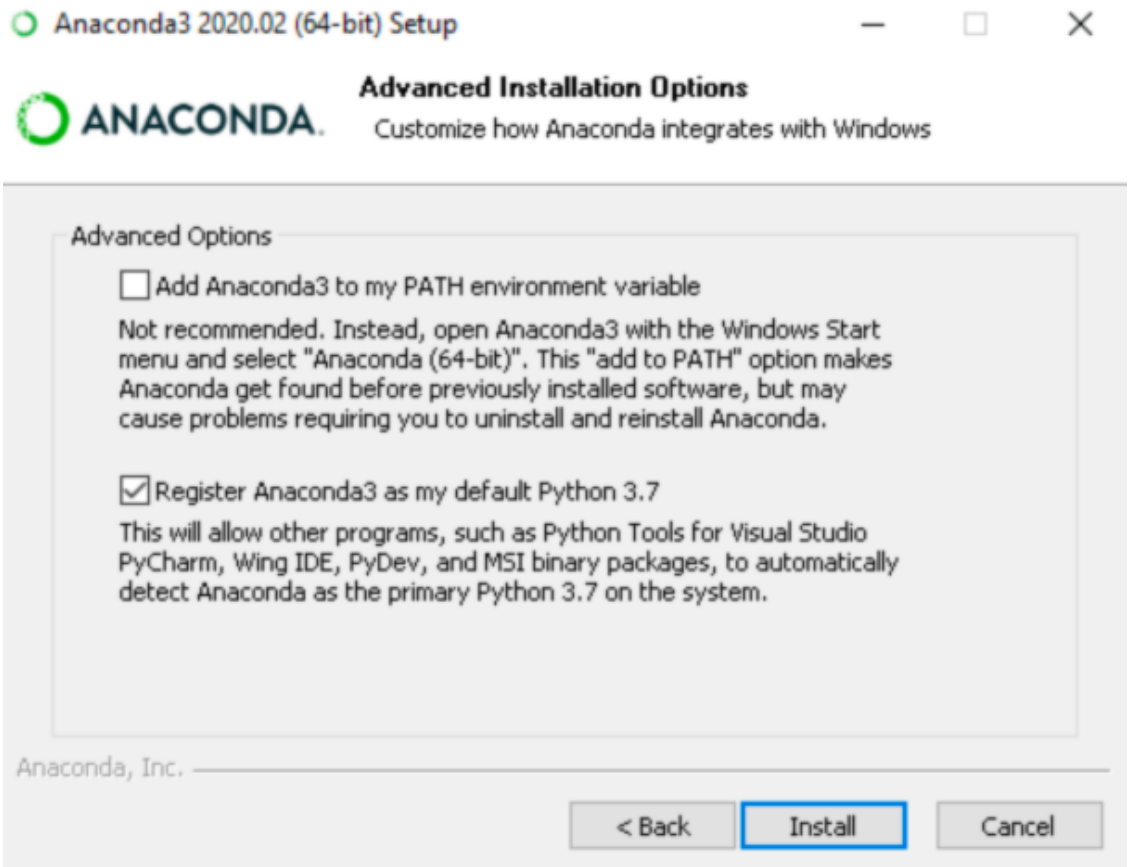
Anaconda es un IDE o distribución **libre** que suele utilizar los lenguajes de Python y R. Se le conoce como IDE ya que cuenta con interfaces de usuario donde se pueden utilizar los lenguajes antes mencionados, estas interfaces son las siguientes:

1. Jupyter Notebook.
2. Jupyter Lab.
3. Spyder.
4. RStudio (depende de la versión de Anaconda).

Por lo tanto, si se instala Anaconda, el lenguaje de Python ya va incluido dentro de la instalación, pero cabe mencionar que si se desea utilizar como un kernel para enlazarlo a otra IDE diferentes de las de Anaconda, tal vez se presenten problemas o se tenga un pooco de dificultad para hacerlo.

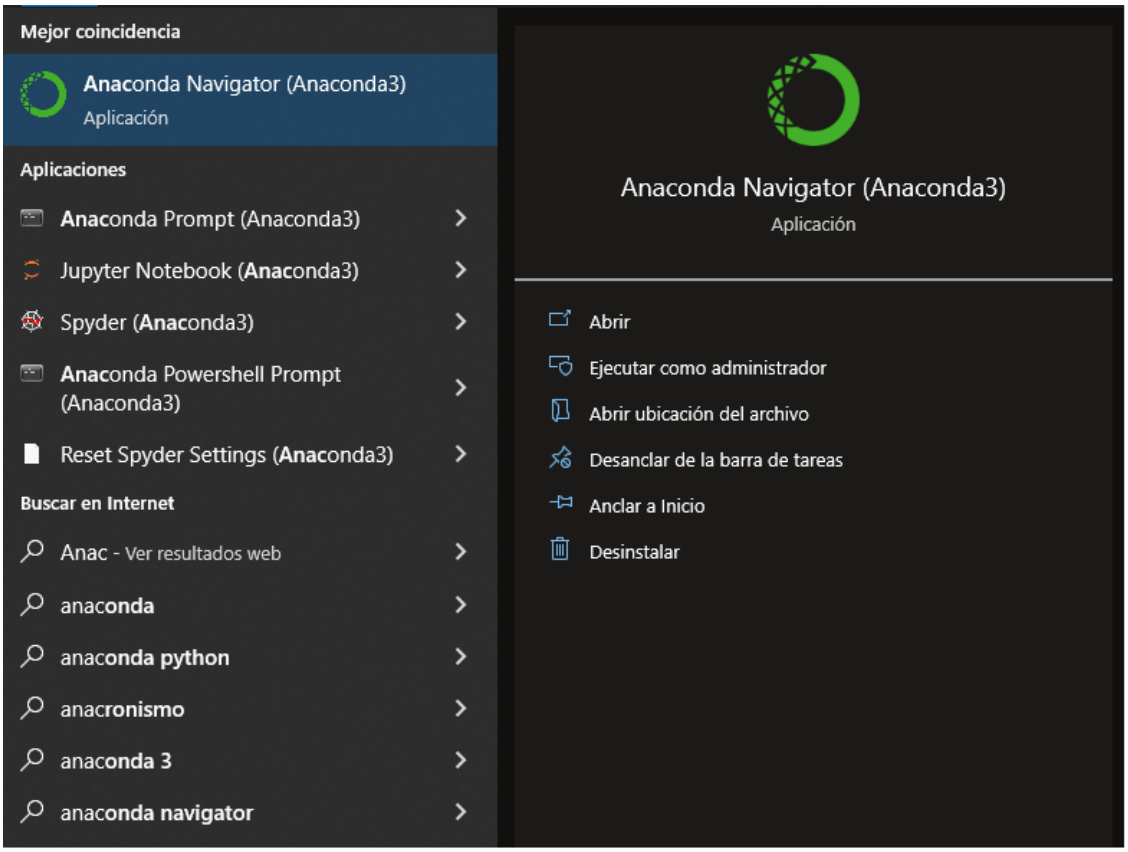
Para realizar la instalación basta con ejecutar el archivo *.exe* de Anaconda como **administrador** para que este se ejecute y se instale correctamente. Solo se debe tener cuidado en la casilla que

se marca en esta parte:

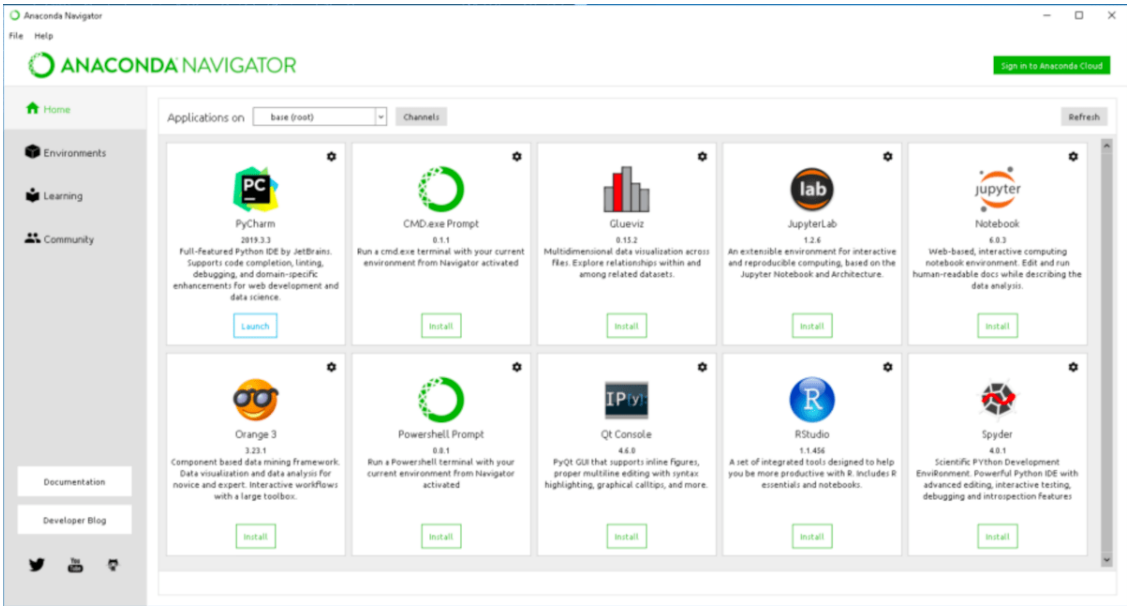


Ya que aquí debemos marcar la casilla que dice **Register Anaconda3 as my default Python x.x** para que no se tenga problemas con Python y las IDE de Anaconda. Posterior a ello, solo se marcan las casillas como *siguiente* y el proceso de instalación quedará finalziado.

Por ultimo, en Windows tendrás un icono similar a este:



Y al ejecutarlo tendrás a la vista las IDE's que puedes ocupar con Python:



4. ¿Cómo se instala una paquetería en Python?

Dentro de Python hay varias maneras de instalar paqueterías, estas maneras son:

1. Mediante la consola CMD: Este proceso solo aplica si se quiere ocupar Python de raíz sin opcupar algun IDE.
2. Mediante Anaconda: Esto proceso aplica solo si se ocupa Python dentro de un IDE para que las paqueterías queden enlazadas al interfaz de usuario.

A continuación se mostrarán ambos procesos.

4.1. Instalacion de paqueterías mediante CMD

Continuando con esta sección, dentro de Python existen librerías que se requieren para la manipulación y visualización de datos. Estas librerías son las siguientes:

1. Matplotlib.
2. Pandas.
3. Seaborn.
4. Plotly.
5. ggplot.

Para poder agregar dichas librerías, se requiere conocer cómo instalar dichos paquetes.

En el caso del lengueje Python, hay que conocer si se tiene disponible el paquete **pip**, en caso de que no, utilizamos CMD para instalarlo mediante el comando **python get-pip.py** y en la consola comenzará el proceso de instalación. Una vez finalizado dicho proceso, se comprueba qué el paquete pip se haya instalado correctamente usando el comando **pip -V** y debe mostrar la versión del pip, tal y como se muestra a continuación:

```
C:\Users\Morales>pip -V
pip 21.2.3 from C:\Users\Morales\AppData\Local\Programs\Python\Python39\lib\site-packages\pip (python 3.9)
```

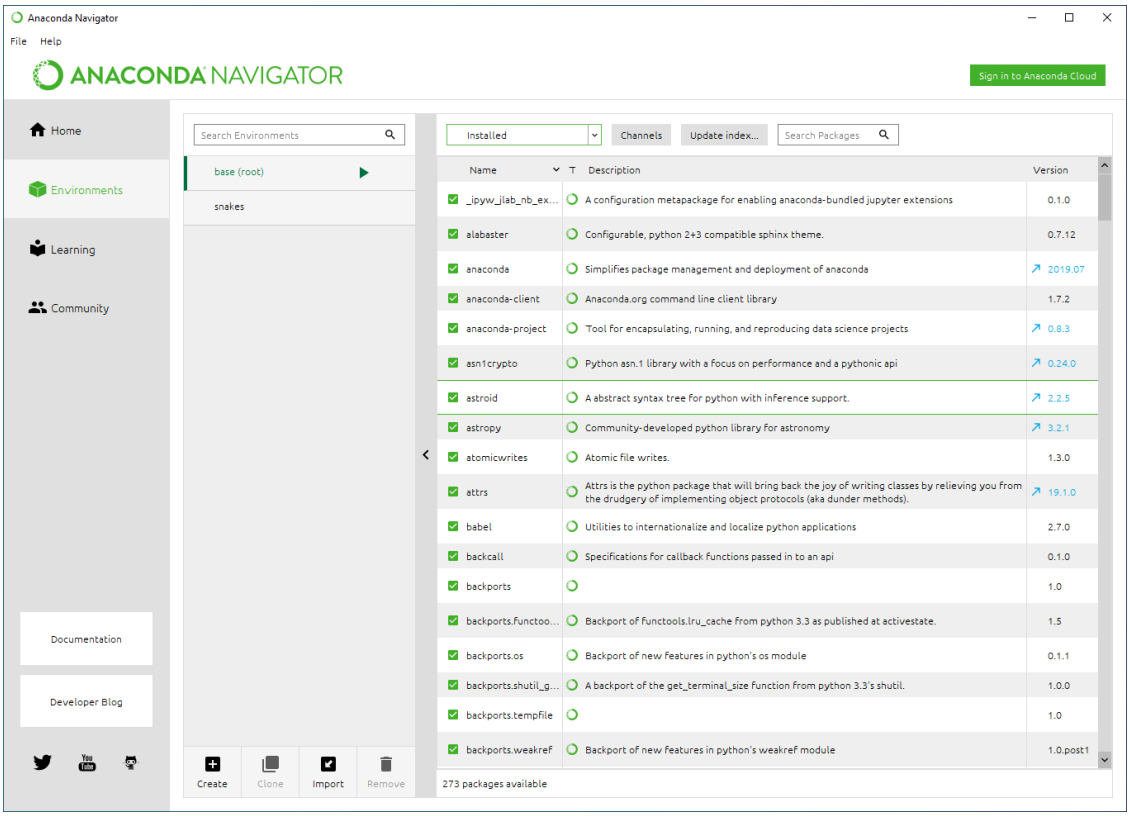
Una vez realizado lo anterior, se puede proceder a reaizar la instalación de los paquetes que se van a ocupar a lo largo de todo el curso. Para ello, se ejecuta la linea en CMD como: **pip install package_name** y con ello se podrá realizar la instalación de paquetes, tal como se muestra en el siguiente ejemplo:

```
C:\Users\Morales>pip install ggplot
Collecting ggplot
  Downloading ggplot-0.11.5-py2.py3-none-any.whl (2.2 MB)
     |████████████████████████████████| 2.2 MB 1.7 MB/s
Requirement already satisfied: matplotlib in c:\users\morales\appdata\local\programs\python\python39\lib\site-packages (from ggplot) (3.4.3)
Requirement already satisfied: numpy in c:\users\morales\appdata\local\programs\python\python39\lib\site-packages (from ggplot) (1.20.3)
Requirement already satisfied: statsmodels in c:\users\morales\appdata\local\programs\python\python39\lib\site-packages (from ggplot) (0.13.0)
Requirement already satisfied: six in c:\users\morales\appdata\local\programs\python\python39\lib\site-packages (from ggplot) (1.16.0)
Collecting brewer2mpl
  Downloading brewer2mpl-1.4.1-py2.py3-none-any.whl (24 kB)
Requirement already satisfied: scipy in c:\users\morales\appdata\local\programs\python\python39\lib\site-packages (from ggplot) (1.7.1)
Requirement already satisfied: cycler in c:\users\morales\appdata\local\programs\python\python39\lib\site-packages (from ggplot) (0.11.0)
Requirement already satisfied: pandas in c:\users\morales\appdata\local\programs\python\python39\lib\site-packages (from ggplot) (1.3.4)
Requirement already satisfied: patsy>=0.4 in c:\users\morales\appdata\local\programs\python\python39\lib\site-packages (from ggplot) (0.5.2)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\morales\appdata\local\programs\python\python39\lib\site-packages (from matplotlib->ggplot) (2.8.2)
Requirement already satisfied: pillow>=6.2.0 in c:\users\morales\appdata\local\programs\python\python39\lib\site-packages (from matplotlib->ggplot) (8.4.0)
Requirement already satisfied: kiwisolver>=1.0.1 in c:\users\morales\appdata\local\programs\python\python39\lib\site-packages (from matplotlib->ggplot) (1.3.2)
Requirement already satisfied: pyparsing>=2.2.1 in c:\users\morales\appdata\local\programs\python\python39\lib\site-packages (from matplotlib->ggplot) (2.4.7)
Requirement already satisfied: pytz>=2017.3 in c:\users\morales\appdata\local\programs\python\python39\lib\site-packages (from pandas->ggplot) (2021.3)
Installing collected packages: brewer2mpl, ggplot
Successfully installed brewer2mpl-1.4.1 ggplot-0.11.5
WARNING: You are using pip version 21.2.3; however, version 21.3.1 is available.
You should consider upgrading via the 'C:\Users\Morales\AppData\Local\Programs\Python\Python39\python.exe -m pip install --upgrade pip' command.
```

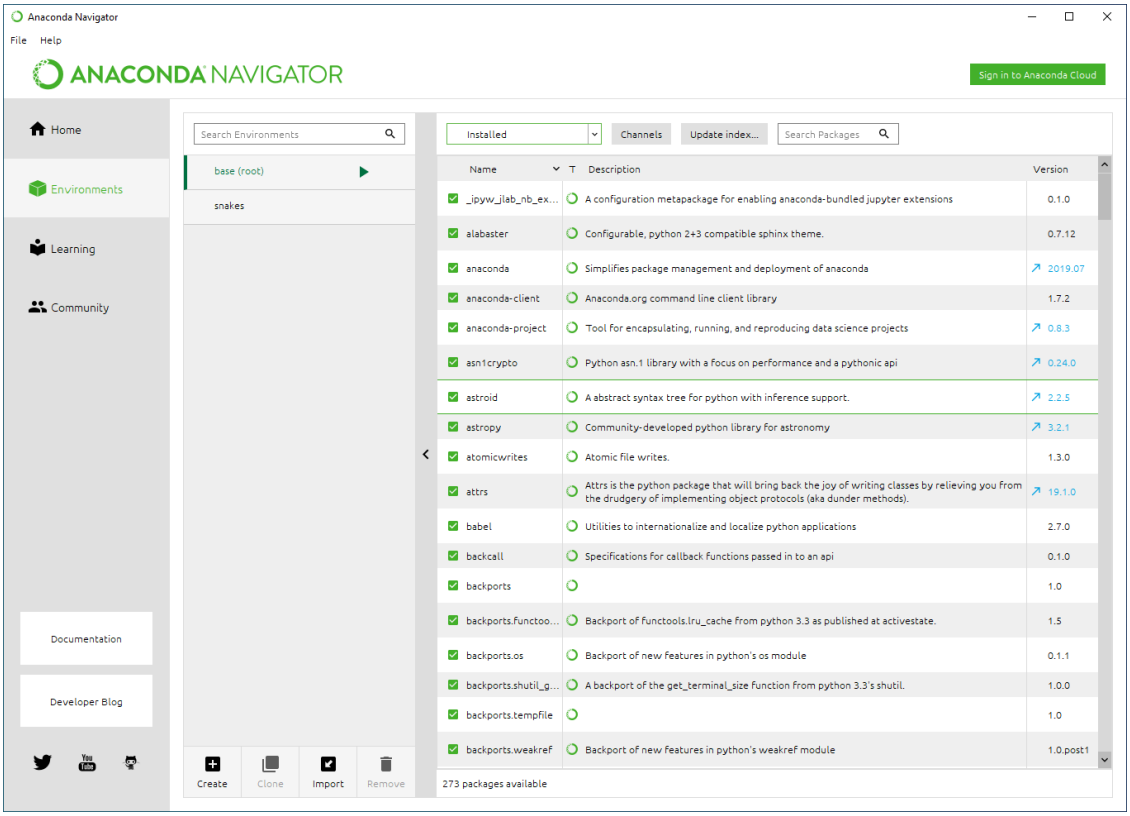
Y con el proceso anterior, el paquete quedó instalado de forma satisfactoria.

4.2. Instalando paquetes mediante Anaconda

Para la instalación de paqueterías dentro de Anaconda, se utilizará las pestaña **Environments** donde se pueden administrar las paqueterías, ya sea instalar, desinstalar o actualizar, tal y cómo se muestra a continuación:

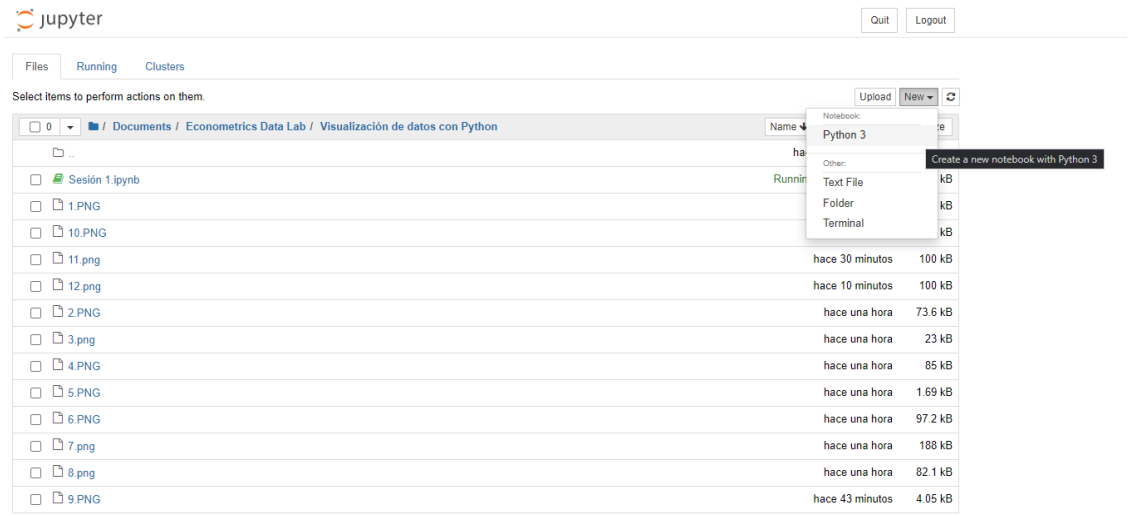


Ahora, si se desea instalar alguna librería, se selecciona la casilla **not installed** y se busca dentro de la lista, las paqueterías que se deseen instalar.

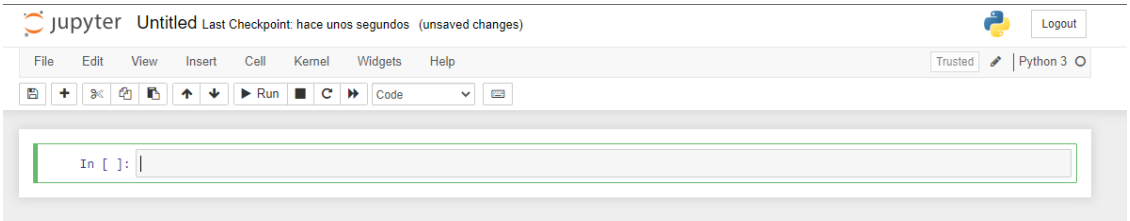


Sin embargo, hay ocasiones en las cuales no se encuentran paquetes en especifico por lo que se debe usar una alternativa para cumplir con el proceso se instalación de librerías. Para ello se recomienda utilizar **Jupyter Notebook**, solo basta con irse a las IDE del menu principal y seleccionar la casilla que tiene por nombre la interfaz.

Una vez dentro de **Jupyter Notebook** se selcciona **New > Python3**, tal y como se muestra a continuación:



Una vez creado el cuaderno se mostrará algo así:



Y dentro de la línea se escribirá: **!pip3 install package_name** y se ejecutará el comando con *shift+Enter* y se ejecutará la instalación, tal y como se muestra a continuación:

```
In [1]: !pip3 install plotly

        Requirement already satisfied: plotly in c:\programdata\anaconda3\lib\site-packages (5.3.1)
        Requirement already satisfied: six in c:\programdata\anaconda3\lib\site-packages (from plotly) (1.15.0)
        Requirement already satisfied: tenacity>=6.2.0 in c:\programdata\anaconda3\lib\site-packages (from plotly) (8.0.1)
```

Y de esta manera los paquetes quedarán instalados dentro de Anaconda.

5. ¿Cómo se importa una librería para trabajar en una sesión?

Dentro de Python, para poder utilizar una librería hay que dar la instrucción **import** ya que se debe importar las funciones de un modulo en su conjunto o solo una función en especifico. Para importar un modulo completo, después de la sentencia de *import* y posterior al nombre del modulo, se utiliza un **as** (como) para que se le asigne un nombre que permita su facil uso dentro de la sesión de Python. A continuación se muestra un ejemplo:

```
[1]: import pandas as pd
```

El resultado de la sentencia anterior no genera una salida, pero lo que se hizo anteriormente tuvo como finalidad importar todo el modulo de **pandas**, entonces,si se requiere utilizar un función de dicho modulo se debe llamar mediante **pd** (por que así se asigno el modulo) seguido de un punto **(.)** con lo cual permitirá el uso de una función en especifico, tal como se muestra a continuación:

```
[2]: ejemplo=pd.Series([1,2,3,4,5])
     ejemplo
```

```
[2]: 0    1
     1    2
     2    3
     3    4
     4    5
     dtype: int64
```

Lo anterior sí genero una salida, ya que e elaboro un objeto mediante la función **Series** perteneciente a la librería **pandas** y al final al llamar al objeto creado, se imprime el resultado.

5.1. Importando una función de un modulo

Para importar una función en especifico de un modulo, se debe tener en cuenta que hay que conocer la **ruta** en donde se encuentra la instrucción a utilizar, para ello, se ocupa **from** seguido

de **import** y se utiliza el nombre de la función, por ejemplo:

[3]:
```
from pandas import Series

ejemplo2=Series([1,2,3,4])
ejemplo2
```

[3]:
```
0    1
1    2
2    3
3    4
dtype: int64
```

6. Operaciones básicas

Python al ser un lenguaje de programación, tiene una sintaxis básica (como todo lenguaje) y dentro del mismo se pueden generar operaciones básicas tales como:

1. Suma.
2. Resta.
3. División.
4. Multiplicación

Es por ello que, a continuación se mostrarán las operaciones básicas que se pueden realizar con Python.

6.1. Suma

Para elaborar una suma, solo se requiere de utilizar el operador (**+**) y realizar el proceso de suma. La codificiación y el resultado no tardará nada ya que el proceso es muy simple. A continuación se muestran 3 ejemplos:

[4]:
```
3+1
```

[4]:
```
4
```

[5]:
```
-5+(-7)
```

[5]:
```
-12
```

[6]:
```
-9+412
```

[6]:
```
403
```

Se puede apreciar, que las leyes matemáticas se respetan y si se incluyen operaciones que provengan de una instrucción matemática, lo respetará y realizará los calculos puntuales.

6.2. Resta

Dentro de las operaciones básicas, la resta se puede realizar mediante el operador (-). Con ello cuando se realice la operarción el lenguaje entenderá que debe realizar el proceso de resta. A continuación se muestran 3 ejemplo:

[7]:
```
2-5
```

[7]: -3

[8]:
```
5-(-11)
```

[8]: 16

[9]:
```
250-51
```

[9]: 199

Como conclusión, las leyes matemáticas se siguen respetando para cada operación realizada.

6.3. Multiplicación

Continuando con las operaciones básicas, la multiplicación se realiza mediante el operador (*). Con dicho símbolo el lenguaje lo entenderá como un producto. A continuación se muestran 3 ejemplos:

[10]:
```
-52*(-89)
```

[10]: 4628

[11]:
```
25*25
```

[11]: 625

[12]:
```
-14*75
```

[12]: -1050

Se aprecia que las condiciones o propiedades matemáticas las sigue respetando.

6.4. División

Para finalizar con esta parte de las operaciones básicas, ahora se mostrará cómo se puede realizar la operación de división. Para ello se debe ocupar el símbolo (/) entre los números deseados, a continuación se muestran 4 ejemplos:

[13]:
```
10/2
```

[13]: 5.0

[14]:
```
-125/5
```

[14]: -25.0

[15]:
```
-5/-125
```

[15]: 0.04

[16]:
```
1/3
```

[16]: 0.3333333333333333

Como conclusión, para el caso de las divisiones, aparte de respetar las leyes de los signos, se aprecia que cuando se tratan de números irracionales, los resultados se expresan con su respectiva extensión decimal.

7. Creación de objetos

En Python, para la creación de objetos solo basta con asignar el operador de igualdad (**=**), donde del lazo izquierdo se definirá el nombre del objeto y del lado derecho el valor que se asigne dicho valor. Para poder verificar que el valor asignado es el correcto se puede hacer mediante **print** y dentro de la sentencia se coloca el nombre del objeto, o bien, solo se puede hacer el llamo del objeto, tal y como se mostrará a continuación:

[17]:
```
x=2
print(x)
```

2

[18]:
```
x=5
x
```

[18]: 5

A medida que se vaya conociendo las funciones de los modulos o librerías de Python, se pueden asignar no solo valores matemáticos o númericos, sino otro tipo de objetos tales como:

- Array.
- Data frame.
- List.
- Tupulas.
- Funciones.
- Etc.

Es por eso que más adelante se explicarán la manera en cómo se pueden declarar esta clase de objetos y cómo identificar a qué clase pertencen una vez elaborados.

8. Tipos de objetos

En este apartado se mostrarán las maneras en las cuales se puede elaborar los diferentes tipos de objetos que se enlistaron anteriormente.

8.1. Array

Para la creación de un array, se debe importar la función **array** del modulo que lleva por mismo nombre y su estructura solo consiste en agregar los elementos que se desean que incluya el objeto dentro de un parentesís (), pero en caso de agregar un lista se debe hacer con un corchete dentro del parentesis ([]), y para verificar que el objeto fue creado correctamente, se utiliza **type** para poder identificar el tipo. A continuación se muestra la elaboración de un array.

[19]:
```
from array import array

objeto=array("i",[1,2,3,4,5])
print(objeto)
```

```
array('i', [1, 2, 3, 4, 5])
```

[20]:
```
type(objeto)
```

[20]:
```
array.array
```

8.2. Data frame

El concepto de data frame es igual a un **marco de datos**, por lo que en la estructura del mismo tiene como objetivo almacenar información mediante columnas y filas, algo similar a una hoja de calculo o un archivo de valores. Para elaborar esta clase de objetos, se debe ocupar la función **DataFrame** de la librería **pandas** y posteriormente para generar columnas y valores se debe utilizar el nombre del dataframe seguido de un chorchete donde dentro del mismo se asginará el nombre de la variable que se desea crear. Al finalizar, se ocupa **type** para verificar la clase del objeto.

A continuación se muestra un ejemplo:

[21]:
```
import pandas as pd

df=pd.DataFrame()

df["Var1"]=(1,2,4,5,6,4,7,8,9,10)
df["Var2"]=("Hombre", "Mujer",
            "Hombre", "Mujer",
            "Hombre", "Mujer",
            "Hombre", "Mujer",
            "Hombre", "Mujer")
df["Var3"]=(1,8,9,1,2,4,8,9,6,4)

df
```

[21]:

	Var1	Var2	Var3
0	1	Hombre	1
1	2	Mujer	8
2	4	Hombre	9

```
3      5   Mujer      1
4      6  Hombre      2
5      4   Mujer      4
6      7  Hombre      8
7      8   Mujer      9
8      9  Hombre      6
9     10   Mujer      4
```

[22]:
```
type(df)
```

[22]:
```
pandas.core.frame.DataFrame
```

Se aprecia que para generar un data frame, conlleva un proceso un poco más largo, pero no complejo por lo que es un objeto muy simple de realizar y al comporbar con **type** se descubre que si es *data frame* que se elaboro mediante *pandas*.

8.3. List

La lista o **list** es un tipo de objeto que almacena una serie de elementos ya sean números, caracteres (letras) o valores lógicos. Para elaborar esta clase de elementos, solo basta con asignar un objetivo y con el uso de corchetes [] se hace el llenado de la lista, tal y como a continuación se muestra:

[23]:
```
lista = ['Juan', 100, False]
lista
```

[23]:
```
['Juan', 100, False]
```

[24]:
```
type(lista)
```

[24]:
```
list
```

Al momento de llamar la función **type** se comrpueba que lo que se elaboro en el ejmplo es correcto y esta en el formato de *lista*.

8.4. Tupulas

Al igual que las listas, el objeto de *tupula* permitirá almacenar valores consecutivos o en secuencia. Pueden ser útiles en situaciones en las que desee compartir los datos con alguien pero no permitir que los manipulen. Sin embargo, se pueden usar los valores de los datos, pero no se refleja ningún cambio en los datos originales compartidos, a continuación se muestra un ejemplo.

[25]:
```
Tupula=(1,2,3,4,5,"Juan", False)
Tupula
```

[25]:
```
(1, 2, 3, 4, 5, 'Juan', False)
```

[26]:
```
type(Tupula)
```

[26]: tuple

La desventaja de la tupula, es que si se desea extraer algún valor de dicho elemento, este no permitirá hacerlo, por lo que se recomienda utilizar más el formato de lista que la tupula.

8.5. Funciones

Una función permite definir un bloque de código reutilizable que se puede ejecutar muchas veces dentro de una sesión. Las funciones pueden crear soluciones más modulares, si bien Python ya proporciona muchas funciones integradas como *print()* y *len()*, también puedes definir tus propias funciones para en otros proyectos.

Una de las grandes ventajas de usar funciones en la sesión es que el códig se reduce en el número total de líneas, la sintaxis básica de Python para crear una función es la siguiente:

- La palabra clave **def** (para definir).
- Nombre de la función.
- Paréntesis '()', y dentro de los paréntesis los parámetros de entrada, aunque los parámetros de entrada sean opcionales.
- Dos puntos ':'
- Algún bloque de código para ejecutar.
- Una sentencia de retorno (opcional).

A continuación se muestra un ejemplo:

[27]:
```
def multiplicacion(x, y):
  return x * y

multiplicacion(45,80)
```

[27]: 3600

[28]:
```
type(multiplicacion)
```

[28]: function

De la anterior manera es como se pueden elaborar los objetos de tipo **función** mediante Python, lo cual puede ser muy beneficiono o no, según sea el caso de aplicación.

8.6. Sustracción de elementos en los diferentes tipos de objetos

Dentro de los objetos existe una manera de poder sustraer los elementos dentro de los mismos, con la finalidad de ocupar solo los valores especificos según el objetivo del ejercicio. Para poder sustraer un elemento de los diferentes tipos de objetos (exceptuando a las tupulas) se utiliza el corchete [] y se escribe el nombre o posición del elemento a sustraer, a continuación se mostrará la manera en la que se pueden sustraer elementos:

[29]:
```
## Para el caso de un array

objeto[1]
```

[29]: 2

[30]:
```
## Para el caso de una lista

lista[0]
```

[30]: 'Juan'

[31]:
```
## Para el caso del dataframe

df["Var3"]
```

[31]:
```
0    1
1    8
2    9
3    1
4    2
5    4
6    8
7    9
8    6
9    4
Name: Var3, dtype: int32
```

El número que va adentro de los corchetes para el caso del arrayt y la lista, tiene que ver con la posición de los elementos, hay que recordar que Python no enumera los objetos de 1 hasta su extensión, sino enumera el 0 como el primer elemento y de ahí comienza la secuencia como: 1 para el segundo elemento, 2 para el tercer elemento y así sucesivamente.

9. Vectores y matrices

Para trabajar con los vectores y matrices en Python, es algo diferente a lo que otros lenguajes hacen. Ya que al tratarse de un lenguje netamente de programación no esta precisamente orientado en las matemáticas o en la estadística. Es por ello que existirán funciones similares a los conceptos matemáticos, tales como el **array** y **list** que son objetos muy similares a una matriz, sin embargo, con la librería **numpy** se podrán realizar matrices y vectores en un formato amigable.

9.1. Generando vectores

Para generar un vector, se puede hacer medianre **array** de **numpy** y ahí se almacenan los valores que se quieren guardar como vector. A continuación se muestra un ejemplo:

[32]:
```
# !pip3 install numpy

import numpy as np

vector=np.array([1,2,3,4,5,6])
vector
```

```
[32]: array([1, 2, 3, 4, 5, 6])
```

También se puede usar la función *arange* para conseguir una secuencia de valores que vayas de 1 en 1, por ejemplo:

```
[33]: vector2=np.arange(1,11)
      vector2
```

```
[33]: array([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10])
```

9.2. Generando matrices

Nuevamente para generar matrices, se utilizará la librería **numpy** pero en esta ocasión se utilizará la función **matrix**. Para poder declarar esta clase de objetos, se debe argumentar como una lista, pero los números deben ir entre comillas dobles o simples, tomando en cuenta que los valores almacenados tomaran formas de filas, pero para ir llenando más filas, se utilizara punto y coma para hacer esta separación y rellenar las filas que se deseen.

A continución se muestra un ejemplo:

```
[34]: A=np.matrix("11,12,13;21,22,23;31,32,33")
      A
```

```
[34]: matrix([[11, 12, 13],
              [21, 22, 23],
              [31, 32, 33]])
```

9.2.1. Operaciones con matrices

A continuación se mostrará la manera de realizar operaciones entre matrices, tales como: Suma, resta, multiplicación, determinante e inversa de matrices. Para ello se generar dos matrices cuadradas (para el caso de suma, resta, determinante y transpuesta) y para el caso de la multiplicación se elaboraran dos matrices no cuadradas.

Para el caso de la **suma** se deben establecer dos matrices cuadradas (en este caso será A y B) y para generación la adición solo se utiliza el operador (**+**) y se procede a ejecutar la instrucción.

```
[35]: ## Suma

      A=np.matrix("4,5,6;-1,7,-8;5,7,6")
      B=np.matrix("8,1,3;-9,4,-7;-2,3,5")

      A+B
```

```
[35]: matrix([[ 12,   6,   9],
              [-10,  11, -15],
              [  3,  10,  11]])
```

Ahora, para el caso de la resta, solo basta con tener dos matrices cuadradas y utilizar el operador (**-**) con ello se realizara la operación entre las matrices.

[36]:
```
## Resta

A-B
```

[36]:
```
matrix([[-4,  4,  3],
        [ 8,  3, -1],
        [ 7,  4,  1]])
```

Si se desea calcular la determinante de una matriz, debemos tener una matriz cuadrada y ocupar una función de **numpy** que tiene por nombre **linalg.det()** y dentro del argumento va el nombre de la matriz, tal y como se muestra a continuación:

[37]:
```
## Determinante

np.linalg.det(A)
```

[37]:
```
-29.999999999999936
```

[38]:
```
## Determinante

np.linalg.det(B)
```

[38]:
```
330.0000000000002
```

Para realizar una transpuesta de la matriz, se debe utilizar el nombre de la matriz seguido de un punto (**.**) y la instrucción **T** que se refiere a la transpuesta de la matriz. Esta clase de funciones ya van implicitas en los objetos de matrices generados por **numpy**, a continuación se muestra un ejemplo:

[39]:
```
## Transpuesta

A.T
```

[39]:
```
matrix([[ 4, -1,  5],
        [ 5,  7,  7],
        [ 6, -8,  6]])
```

En el caso de la multiplicación, se debe contar con matrices cuadradas y con ello utilizar el operador (*) que permite realizar las multiplicaciones tanto de números como de matrices, tal y como se muestra a continuación:

[40]:
```
## Multiplicación

C=np.matrix("4,5,6;5,7,6")
D=np.matrix("8,1;-2,5;5,9")

C*D
```

[40]:
```
matrix([[52, 83],
        [56, 94]])
```

[41]:
```
## Multiplicación

D*C
```

[41]:
```
matrix([[37, 47, 54],
        [17, 25, 18],
        [65, 88, 84]])
```

Pos último, para realiza o calcular la inversa de una matriz, se utilizará una función de **numpy** que tiene por nombre **linalg.inv()** en la cual dentro del argumento se escribe el nombre de la matriz y con ello se pueden generar la operación. Hay que recordar que para este proceso es necesario contar con matrices cuadradas tal y como se muestra a continuación:

[42]:
```
## Inversa

np.linalg.inv(A)
```

[42]:
```
matrix([[-3.26666667, -0.4       ,  2.73333333],
        [ 1.13333333,  0.2       , -0.86666667],
        [ 1.4       ,  0.1       , -1.1       ]])
```

[43]:
```
## Inversa

np.linalg.inv(B)
```

[43]:
```
matrix([[ 0.12424242,  0.01212121, -0.05757576],
        [ 0.17878788,  0.13939394,  0.08787879],
        [-0.05757576, -0.07878788,  0.12424242]])
```